

AMS 580 Quiz 6 - RNN and LSTM Solution

```
library(keras3)
library(ggplot2)
library(dplyr)
library(readr)
library(tidyr)

set.seed(123)
try(keras3::set_random_seed(123), silent = TRUE)

make_sequences <- function(series, seq_len = 5) {
  n <- length(series)
  X <- array(NA_real_, dim = c(n - seq_len, seq_len, 1))
  y <- numeric(n - seq_len)
  for (i in 1:(n - seq_len)) {
    X[i, , 1] <- series[i:(i + seq_len - 1)]
    y[i] <- series[i + seq_len]
  }
  list(X = X, y = y)
}

recursive_forecast <- function(model, last_sequence, h = 20) {
  preds <- numeric(h)
  current_seq <- as.numeric(last_sequence)
  for (i in 1:h) {
    x_input <- array(current_seq, dim = c(1, length(current_seq), 1))
    next_pred <- as.numeric(predict(model, x_input, verbose = 0))
    preds[i] <- next_pred
    current_seq <- c(current_seq[-1], next_pred)
  }
  preds
}

inverse_z <- function(x_scaled, train_mean, train_sd) {
  x_scaled * train_sd + train_mean
}
```

Task. RNN and LSTM Forecasting for JPM Daily Returns

Q1. Read the data and plot JPM daily returns

```
jpm <- read_csv("JPM_lstm_quiz6.csv", show_col_types = FALSE) |>
  mutate(Date = as.Date(Date)) |>
  arrange(Date) |>
```

```
select(Date, Return) |>
na.omit()
```

```
nrow(jpm)
```

```
## [1] 3873
```

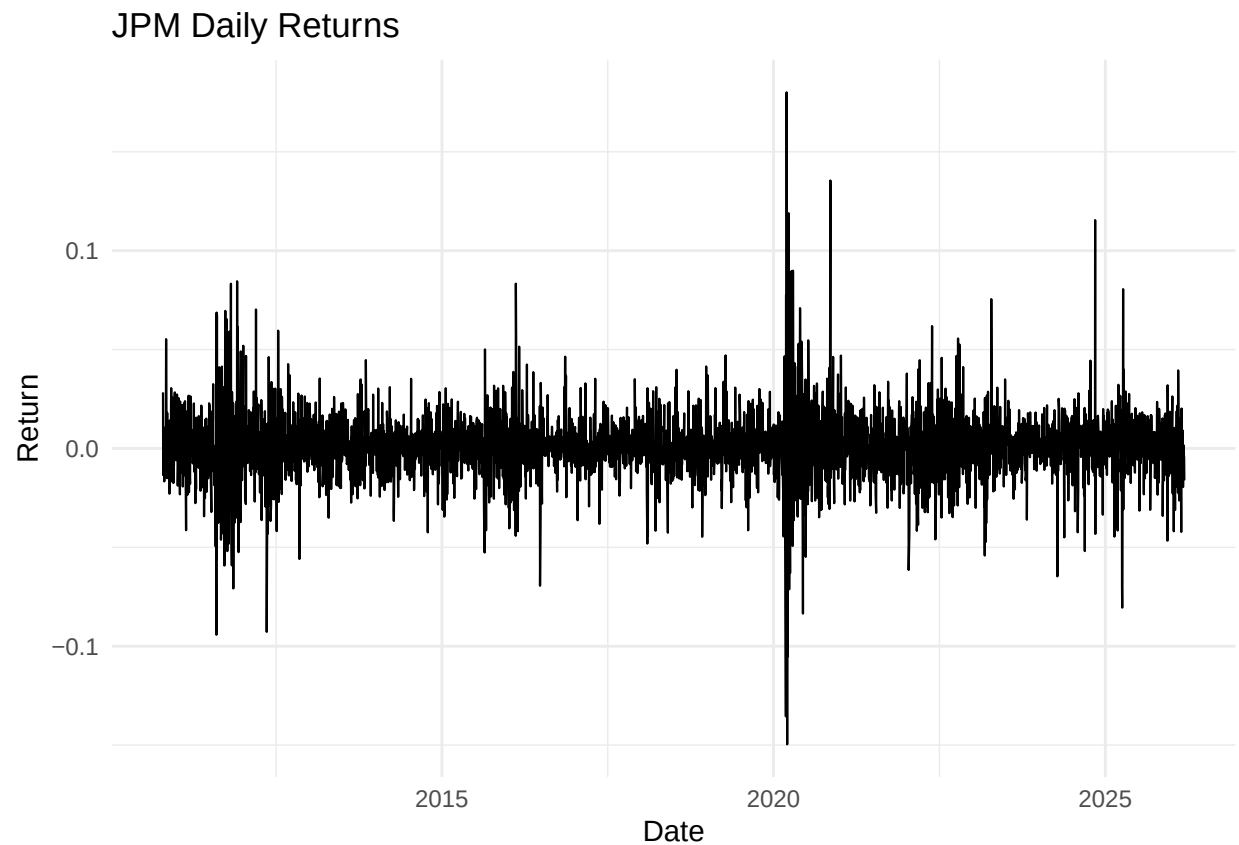
```
head(jpm)
```

```
## # A tibble: 6 x 2
##   Date      Return
##   <date>     <dbl>
## 1 2010-10-18  0.0284
## 2 2010-10-19 -0.0135
## 3 2010-10-20  0.0108
## 4 2010-10-21 -0.0105
## 5 2010-10-22  0
## 6 2010-10-25 -0.0167
```

```
tail(jpm)
```

```
## # A tibble: 6 x 2
##   Date      Return
##   <date>     <dbl>
## 1 2026-03-05 -0.0195
## 2 2026-03-06 -0.0139
## 3 2026-03-09  0.00152
## 4 2026-03-10 -0.00410
## 5 2026-03-11 -0.00419
## 6 2026-03-12 -0.0161
```

```
ggplot(jpm, aes(x = Date, y = Return)) +
  geom_line(linewidth = 0.4) +
  labs(
    title = "JPM Daily Returns",
    x = "Date",
    y = "Return"
  ) +
  theme_minimal()
```



The return series fluctuates around zero. There is no obvious long-term trend in the mean level, although the volatility changes over time.

Q2. Training and testing split

```
test_size <- 20
return_series <- jpm$Return
n <- length(return_series)

train_return <- return_series[1:(n - test_size)]
test_return <- return_series[(n - test_size + 1):n]
test_dates <- jpm$Date[(n - test_size + 1):n]

length(train_return)
```

```
## [1] 3853
```

```
length(test_return)
```

```
## [1] 20
```

Q3. Standardize returns using training mean and training standard deviation

```
train_mean <- mean(train_return)
train_sd <- sd(train_return)

train_scaled <- (train_return - train_mean) / train_sd
test_scaled <- (test_return - train_mean) / train_sd

round(tail(train_scaled, 5), 4)
```

```
## [1] -1.3447  2.2424 -0.0995 -0.7331 -1.4044
```

```
round(head(test_scaled, 5), 4)
```

```
## [1] -1.5713 -0.0627  0.8321  0.2660 -0.1826
```

Q4. Create sliding-window sequences with sequence length 5

```
seq_len <- 5
train_seq <- make_sequences(train_scaled, seq_len = seq_len)

X_train <- train_seq$X
y_train <- train_seq$y

dim(X_train)
```

```
## [1] 3848    5    1
```

```
length(y_train)
```

```
## [1] 3848
```

Q5. Fit the vanilla RNN model

```
model_rnn <- keras_model_sequential(
  layers = list(
    layer_input(shape = c(seq_len, 1)),
    layer_simple_rnn(units = 32),
    layer_dense(units = 1)
  )
)

model_rnn |>
  compile(
    optimizer = "adam",
    loss = "mse"
```

```

)
history_rnn <- model_rnn |>
  fit(
    x = X_train,
    y = y_train,
    epochs = 40,
    batch_size = 32,
    validation_split = 0.1,
    verbose = 0
  )

summary(model_rnn)

```

```

## Model: "sequential"
##
##   Layer (type)                Output Shape          Param #
##   -----
##   simple_rnn (SimpleRNN)      (None, 32)           1,088
##   dense (Dense)               (None, 1)            33
##
## Total params: 3,365 (13.15 KB)
## Trainable params: 1,121 (4.38 KB)
## Non-trainable params: 0 (0.00 B)
## Optimizer params: 2,244 (8.77 KB)

```

Q6. Fit the LSTM model

```

model_lstm <- keras_model_sequential(
  layers = list(
    layer_input(shape = c(seq_len, 1)),
    layer_lstm(units = 32),
    layer_dense(units = 1)
  )
)

model_lstm |>
  compile(
    optimizer = "adam",
    loss = "mse"
  )

history_lstm <- model_lstm |>
  fit(
    x = X_train,
    y = y_train,
    epochs = 40,
    batch_size = 32,
    validation_split = 0.1,
    verbose = 0
  )

```

```
summary(model_lstm)
```

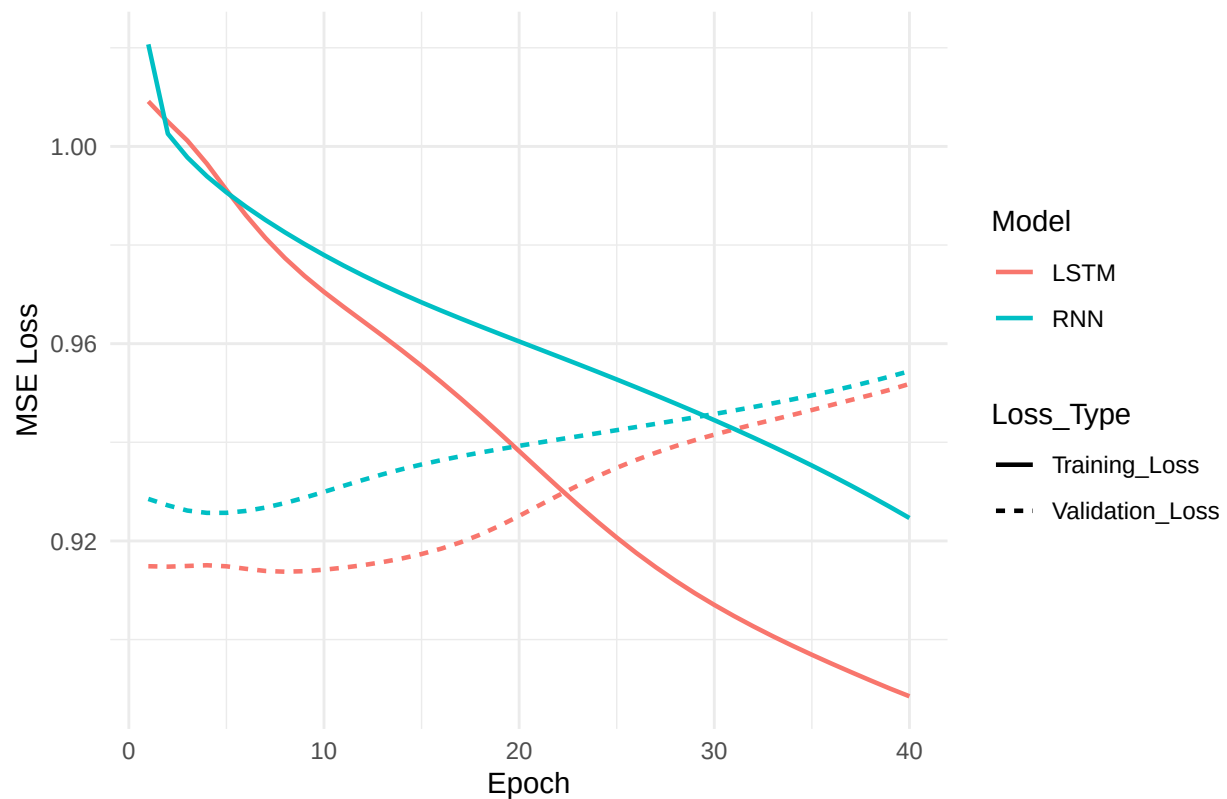
```
## Model: "sequential_1"
##
##   Layer (type)                Output Shape          Param #
##   ---
##   lstm (LSTM)                 (None, 32)            4,352
##   dense_1 (Dense)            (None, 1)             33
##
## Total params: 13,157 (51.40 KB)
## Trainable params: 4,385 (17.13 KB)
## Non-trainable params: 0 (0.00 B)
## Optimizer params: 8,772 (34.27 KB)
```

Q7. Plot training and validation loss

```
loss_df <- bind_rows(
  data.frame(
    epoch = 1:40,
    Model = "RNN",
    Training_Loss = as.numeric(history_rnn$metrics$loss),
    Validation_Loss = as.numeric(history_rnn$metrics$val_loss)
  ),
  data.frame(
    epoch = 1:40,
    Model = "LSTM",
    Training_Loss = as.numeric(history_lstm$metrics$loss),
    Validation_Loss = as.numeric(history_lstm$metrics$val_loss)
  )
) |>
pivot_longer(
  cols = c(Training_Loss, Validation_Loss),
  names_to = "Loss_Type",
  values_to = "Loss"
)

ggplot(loss_df, aes(x = epoch, y = Loss, color = Model, linetype = Loss_Type)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Training and Validation Loss: RNN vs LSTM",
    x = "Epoch",
    y = "MSE Loss"
  ) +
  theme_minimal()
```

Training and Validation Loss: RNN vs LSTM



Both models usually show decreasing or stabilizing loss after several epochs. The validation loss may fluctuate because stock returns are noisy and difficult to predict.

Q8. Recursive 20-day forecasts, inverse scaling, MSE, and plot

```
last_train_seq <- tail(train_scaled, seq_len)

pred_rnn_scaled <- recursive_forecast(model_rnn, last_train_seq, h = test_size)
pred_lstm_scaled <- recursive_forecast(model_lstm, last_train_seq, h = test_size)

pred_rnn <- inverse_z(pred_rnn_scaled, train_mean, train_sd)
pred_lstm <- inverse_z(pred_lstm_scaled, train_mean, train_sd)

mse_rnn <- mean((pred_rnn - test_return)^2)
mse_lstm <- mean((pred_lstm - test_return)^2)

mse_rnn
```

```
## [1] 0.000221439
```

```
mse_lstm
```

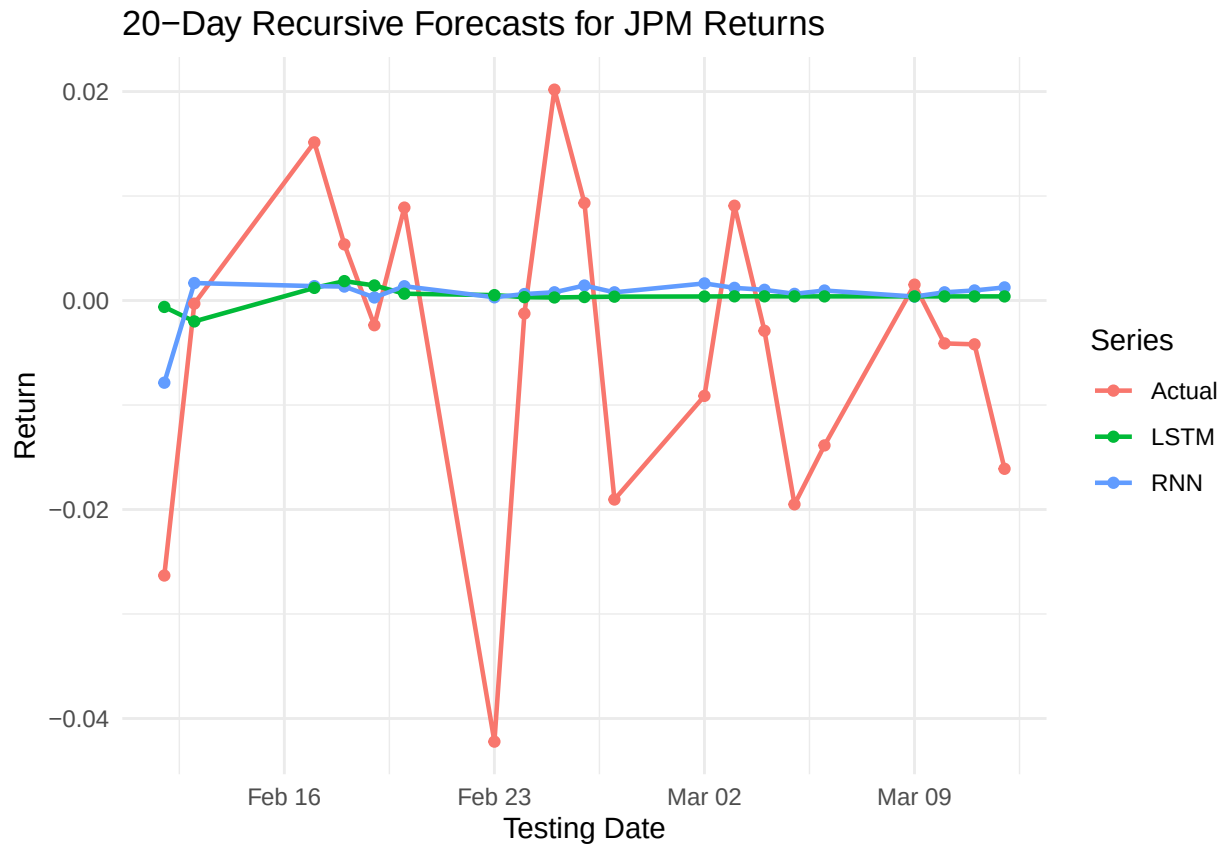
```
## [1] 0.0002362597
```

```

pred_df <- data.frame(
  Date = test_dates,
  Actual = test_return,
  RNN = pred_rnn,
  LSTM = pred_lstm
) |>
  pivot_longer(cols = c(Actual, RNN, LSTM), names_to = "Series", values_to = "Return")

ggplot(pred_df, aes(x = Date, y = Return, color = Series)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.5) +
  labs(
    title = "20-Day Recursive Forecasts for JPM Returns",
    x = "Testing Date",
    y = "Return"
  ) +
  theme_minimal()

```



```

comparison <- data.frame(
  Model = c("RNN", "LSTM"),
  Test_MSE = c(mse_rnn, mse_lstm)
)
comparison

```

```
##   Model   Test_MSE
```



```
## 1   RNN 0.0002214390
## 2   LSTM 0.0002362597
```

Here RNN is better.